

when algorithms dramatically slow down, so recognizing your code as $O(n^2)$ or higher should give you pause. Perhaps there's a different algorithm that can solve the problem faster. In these cases, taking a data structure and algorithms (DSA) course, whether at a university or online, can be helpful.

We also call $O(n^2)$ *quadratic time*. Algorithms could have $O(n^3)$, or *cubic time*, which is slower than $O(n^2)$; $O(n^4)$, or *quartic time*, which is slower than $O(n^3)$; or other polynomial time complexities.

$O(2^n)$, Exponential Time

Taking photos of the shelf with every possible combination of books on it is an exponential time operation. Think of it this way: each book on the shelf can either be included in the photo or not included. [Figure 13-1](#) shows every combination where n is 1, 2, or 3. If n is 1, there are two possible photos: with the book and without. If n is 2, there are four possible photos: both books on the shelf, both books off, the first on and second off, and the second on and first off. When you add a third book, you've once again doubled the amount of work you have to do: you need to do every subset of two books that includes the third book (four photos) and every subset of two books that excludes the third book (another four photos, for 2^3 or eight photos). Each additional book doubles the workload. For n books, the number of photos you need to take (that is, the amount of work you need to do) is 2^n .